Reconstrucción 3D y estimación métrica por estéreo multivista

Trabajo Práctico II

Agustín Patruno Carranza · **Catalina Hirsch** · **Mathias Kneipp**

16 de octubre de 2026

Resumen

1. Introducción y Marco Teórico

El objetivo de este trabajo es reconstruir la nube de puntos 3D de un objeto a partir de un conjunto de imágenes tomadas desde distintos ángulos con una cámara estéreo, y estimar sus dimensiones reales en milímetros. Se utiliza un sistema estéreo porque permite recuperar la profundidad de los píxeles y porque reduce la búsqueda de correspondencias a una recta (la línea epipolar). El pipeline consta de: calibración del par estéreo (intrínsecos, distorsión y extrínsecos relativos R, t), rectificación de las imágenes para alinear las líneas epipolares, cálculo de disparidad para obtener una nube de puntos local por par, estimación de la pose global de cada cámara para fusionar todas las nubes en un marco común, y finalmente optimización de la nube y medición métrica del objeto.

1.1. Modelo de cámara pinhole y distorsión

El modelo pinhole describe cómo una cámara forma una imagen: todos los rayos de luz pasan por un único punto y se proyectan sobre el plano imagen. En esta proyección de perspectiva se pierde la profundidad, ya que un punto del plano puede provenir de infinitos puntos 3D sobre una recta. La proyección de un punto tridimensional sobre la imagen se expresa como

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & \mathbf{t} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}, \tag{1}$$

donde λ es un factor de escala proyectivo, K es la matriz de parámetros intrínsecos, y R y $\mathbf{t}$ representan la rotación y la traslación que transforman las coordenadas del sistema de referencia del mundo al sistema de referencia de la cámara.

La matriz K tiene la forma

$$K = \begin{bmatrix} f_x & \gamma & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}. \tag{2}$$

Aquí, f_x y f_y son las distancias focales expresadas en píxeles en las direcciones horizontal y vertical, respectivamente; c_x y c_y son las coordenadas del punto principal, donde el eje óptico intersecta el plano imagen; y γ representa el parámetro de sesgo (*skew*) entre los ejes de la imagen, que habitualmente se asume nulo.

El modelo pinhole ideal no contempla que las lentes reales deforman la imagen. Esto se corrige con los coeficientes de distorsión: la *radial* (k_1, k_2, k_3) arquea las líneas rectas (efecto barril o cojín), notoria en lentes de cámara web como la de este trabajo, y la *tangencial* (p_1, p_2), por desalineación entre lente y sensor.

Tanto K como los coeficientes de distorsión son desconocidos y propios de cada cámara. Sin ellos no se puede relacionar los píxeles con la geometría real ni obtener medidas en milímetros. Por eso es necesario calibrar: estimarlos a partir de un patrón de geometría conocida. Es el cimiento métrico del pipeline, ya que un error en esta etapa se propaga al resto.

1.2. Calibración estéreo

Se estiman la rotación (R) y traslación (t) relativas entre las dos cámaras (de la derecha respecto de la izquierda). Definen la geometría del par y permiten rectificar y recuperar la profundidad: el módulo de t es la línea base (*baseline* B), que entra en $Z = f\,B/d$.

Para estimarlos se toman pares de imágenes de un patrón (checkerboard/ChArUco) de geometría conocida vistos por ambas cámaras a la vez, se detectan sus esquinas y se resuelve una optimización (`stereoCalibrate`) que halla el R, t consistente con todas las vistas (y afina intrínsecos y distorsión).

La calidad se mide con el error de reproyección: se reproyectan los puntos 3D del patrón al plano imagen con los parámetros estimados y se mide, en píxeles, la distancia a las esquinas detectadas, reportada como raíz cuadrática media (RMS); cuanto más bajo, mejor.

$$e_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2} \tag{3}$$

donde $\mathbf{x}_i$ es la esquina detectada, $\hat{\mathbf{x}}_i$ su reproyección a partir del punto 3D conocido, y N el total de puntos sobre todas las imágenes.

1.3. Geometría epipolar y rectificación

Dado un punto en la imagen izquierda, su correspondiente en la derecha está obligado a caer sobre una recta, la línea epipolar: el punto 3D está sobre el rayo que sale de la cámara izquierda, y ese rayo se proyecta como una recta en la derecha. Así, la búsqueda de correspondencias pasa de 2D a 1D. Esta restricción la captura la matriz fundamental F (o la esencial E):

$$\mathbf{x}'^{T} F \mathbf{x} = 0 \tag{4}$$

En general esas líneas epipolares están inclinadas y a distintas alturas en cada imagen. La rectificación reproyecta (homografía) ambas imágenes a un par ideal con planos imagen coplanares, de modo que las líneas epipolares queden horizontales y alineadas en la misma fila.

Tras rectificar, el correspondiente de un píxel está en la misma fila de la otra imagen, desplazado solo horizontalmente. Ese desplazamiento es la disparidad,

$$d = x_{\mathrm{izq}} - x_{\mathrm{der}} \tag{5}$$

Así, el cálculo de disparidad se reduce a una búsqueda 1D a lo largo de la fila, mucho más rápida y robusta que buscar sobre rectas oblicuas distintas para cada píxel.

1.4. De disparidad a profundidad

En el par rectificado, la profundidad Z de un punto se relaciona con su disparidad d por triángulos semejantes mediante

$$Z = \frac{f\,B}{d} \tag{6}$$

donde f es la distancia focal en píxeles y B la línea base. La relación es inversa, por lo que un mismo error en d afecta más a los puntos lejanos y el estéreo es más preciso de cerca. Como B proviene de la calibración en milímetros, Z queda en unidades métricas, sin la ambigüedad de escala de la reconstrucción monocular. Con la profundidad de cada píxel, la matriz de reproyección Q retroproyecta cada píxel (u, v) a un punto 3D (X, Y, Z), formando la nube local.

2. Metodología

El pipeline implementado parte de la calibración y rectificación del par estéreo, calcula la disparidad de cada par y la retroproyecta a una nube de puntos local, estima la pose global de cada cámara respecto de un marco de referencia común para fusionar todas las nubes, y finalmente optimiza la nube resultante y mide el objeto. A continuación se describe cada etapa con las técnicas y parámetros utilizados.

2.1. Calibración y rectificación

La calibración se realizó con la herramienta `calib-stereo` de la cátedra, a partir de 63 pares de un *checkerboard* de 9×6 esquinas internas y cuadrados de 28 mm, capturados desde distintos ángulos. Mediante `calibrateCamera` y `stereoCalibrate` se estiman los intrínsecos y la distorsión de cada cámara y los extrínsecos relativos (R, t); el tamaño físico del cuadrado fija la escala métrica. Los resultados se exportan a archivos `.pkl` que el resto del pipeline consume sin recalibrar.

Con esos parámetros se calcularon los mapas de rectificación (`cv.stereoRectify`, que devuelve además la matriz Q para pasar de disparidad a 3D, e `cv.initUndistortRectifyMap`) y se aplicaron a cada par con `cv.remap`. La alineación se verificó visualmente con líneas horizontales sobre un par rectificado.

2.2. Disparidad y nube local

Para cada par de imágenes estéreo, se aplicaron los mapas de rectificación para alinear ambas cámaras y facilitar la búsqueda de correspondencias sobre las mismas filas. Sobre las imágenes rectificadas en escala de grises se calculó la disparidad mediante `StereoSGBM`, utilizando `blockSize`= 11, `numDisparities`= 240, $P_1 = 8{\cdot}11^2$ y $P_2 = 32{\cdot}11^2$, junto con `uniquenessRatio`= 5, `speckleWindowSize`= 200 y `speckleRange`= 2. Para reducir correspondencias erróneas, se verificó la consistencia entre las disparidades izquierda-derecha y derecha-izquierda, con una tolerancia de 1,5 píxeles. También se aplicó el filtro WLS (*Weighted Least Squares*) para suavizar el mapa de disparidad preservando las discontinuidades de profundidad.

Mediante `cv.reprojectImageTo3D` y la matriz Q obtenida durante la calibración estéreo, las disparidades se reproyectaron al espacio tridimensional para recuperar las coordenadas (X, Y, Z) en el sistema de referencia de la cámara izquierda rectificada. Para construir la nube local, se asignó a cada punto el color correspondiente de la imagen izquierda rectificada. Finalmente, se conservaron los puntos con disparidad superior al mínimo configurado, coordenadas finitas y profundidad dentro del intervalo $100 < Z < 2000$ mm.

2.3. Pose global y transformación

Para expresar las nubes locales en un sistema de referencia común, se estimó la pose de la cámara izquierda respecto del tablero ChArUco. Se detectaron sus esquinas en cada imagen izquierda rectificada y, mediante `board.matchImagePoints`, se obtuvieron las correspondencias entre sus coordenadas en píxeles y sus posiciones tridimensionales conocidas. La estimación se realizó con `cv.solvePnP`, utilizando la matriz intrínseca rectificada, distorsión nula y el método

`SOLVEPNP_SQPNP`. Se exigieron al menos seis esquinas válidas para aceptar la estimación.

La función devuelve un vector de rotación $\mathbf{r}$ y un vector de traslación $\mathbf{t}$, que describen la transformación del sistema de coordenadas del mundo al de la cámara. El vector de Rodrigues se convirtió en una matriz de rotación R mediante `cv.Rodrigues`. La transformación estimada cumple:

$$\mathbf{p}_c = R\mathbf{p}_w + \mathbf{t},$$

donde $\mathbf{p}_w$ y $\mathbf{p}_c$ representan un mismo punto en coordenadas de mundo y de cámara, respectivamente.

Como las nubes locales están expresadas en coordenadas de cámara, se invirtió esta transformación para llevarlas al sistema global. Utilizando $R^{-1} = R^T$, se obtuvo:

$$R_{cw} = R^T, \qquad \mathbf{t}_{cw} = -R^T\mathbf{t}.$$

Ambos componentes se combinaron en la matriz homogénea:

$$T_{cw} = \begin{bmatrix} R^T & -R^T\mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix}.$$

Esta matriz permitió transformar cada nube local al sistema global para su posterior fusión. Los pares cuya imagen no pudo leerse o cuya detección no alcanzó el mínimo de esquinas requerido se descartaron.

2.4. Fusión y optimización en Open3D

Las nubes locales transformadas se fusionaron mediante Open3D. Para controlar el costo computacional y la cantidad de puntos, se aplicó `voxel_down_sample` con un tamaño de vóxel de 2 mm a cada nube local y a la nube acumulada después de cada par procesado (`merge_every`= 1).

Como postprocesamiento de la nube global del Palacio, se delimitó manualmente el objeto mediante una caja alineada con los ejes (*Axis-Aligned Bounding Box*, AABB), con límites $\mathbf{b}_{\text{mín}} = (130, 90, -140)$ mm y $\mathbf{b}_{\text{máx}} = (220, 470, 0)$ mm. Estos límites se definieron a partir de la inspección de la nube global. Sobre la región seleccionada se aplicó una nueva reducción por vóxeles de 1,5 mm y se eliminaron valores atípicos mediante `remove_statistical_outlier`, utilizando `nb_neighbors`= 20 y `std_ratio`= 1,5. Se registró la cantidad de puntos después de cada operación y se calcularon las dimensiones de la nube resultante mediante una AABB y una caja envolvente orientada (*Oriented Bounding Box*, OBB).

2.5. Medición métrica

Las dimensiones del objeto se estimaron a partir de las extensiones de las cajas AABB y OBB calculadas sobre la nube final. La AABB mide las dimensiones respecto de los ejes globales, mientras que la OBB permite considerar la orientación de la nube para obtener una estimación complementaria del tamaño del objeto.

La reconstrucción permite obtener medidas métricas porque la matriz de reproyección Q utiliza los parámetros de la calibración estéreo, incluida la separación entre las cámaras, para recuperar coordenadas tridimensionales a escala física. Por su parte, la pose global se estima a partir de las dimensiones conocidas del tablero ChArUco, configurado con cuadrados de 42 mm de lado. Para que las nubes locales y las poses sean compatibles, las unidades utilizadas en ambas calibraciones deben ser consistentes. Las dimensiones obtenidas se interpretan como estimaciones del tamaño del objeto y su precisión depende de la calibración, la calidad de las disparidades, la estimación de las poses y los filtros aplicados.

3. Resultados y Discusión

3.1. Resultados

3.2. Discusión

Referencias

[1] R. Hartley y A. Zisserman. Multiple View Geometry in Computer Vision. 2.ª ed., Cambridge University Press, 2004.

[2] OpenCV. Camera Calibration and 3D Reconstruction. Documentación oficial.

[3] Q.-Y. Zhou, J. Park y V. Koltun. Open3D: A Modern Library for 3D Data Processing. arXiv:1801.09847, 2018.